Service Layers Explained

Coordinating Business Logic in Apex

FFLib Series · Session 002

Andrew Fawcett

Code With Sally

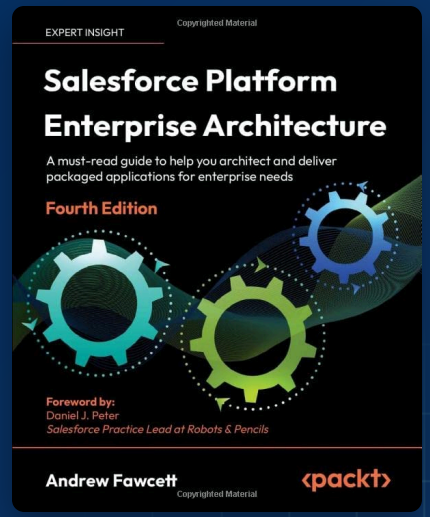
About me

Independent Salesforce Consultant | Former CPO, Salesforce, Heroku | CTO, FinancialForce.com

- **Independent consultant and architect** — Salesforce platform development, PaaS integrations, and agentic AI
- **Open source and guidance** — creator of DLRS, Apex frameworks, and *Salesforce Platform Enterprise Architecture*
- **Salesforce Customers and Partners Advisory** — architecture and scale, AI-enhanced workflows, org health and codebase mentoring, and ISV product advisory

Reduce complexity, unblock engineering, and leave architectures — and teams — able to evolve.

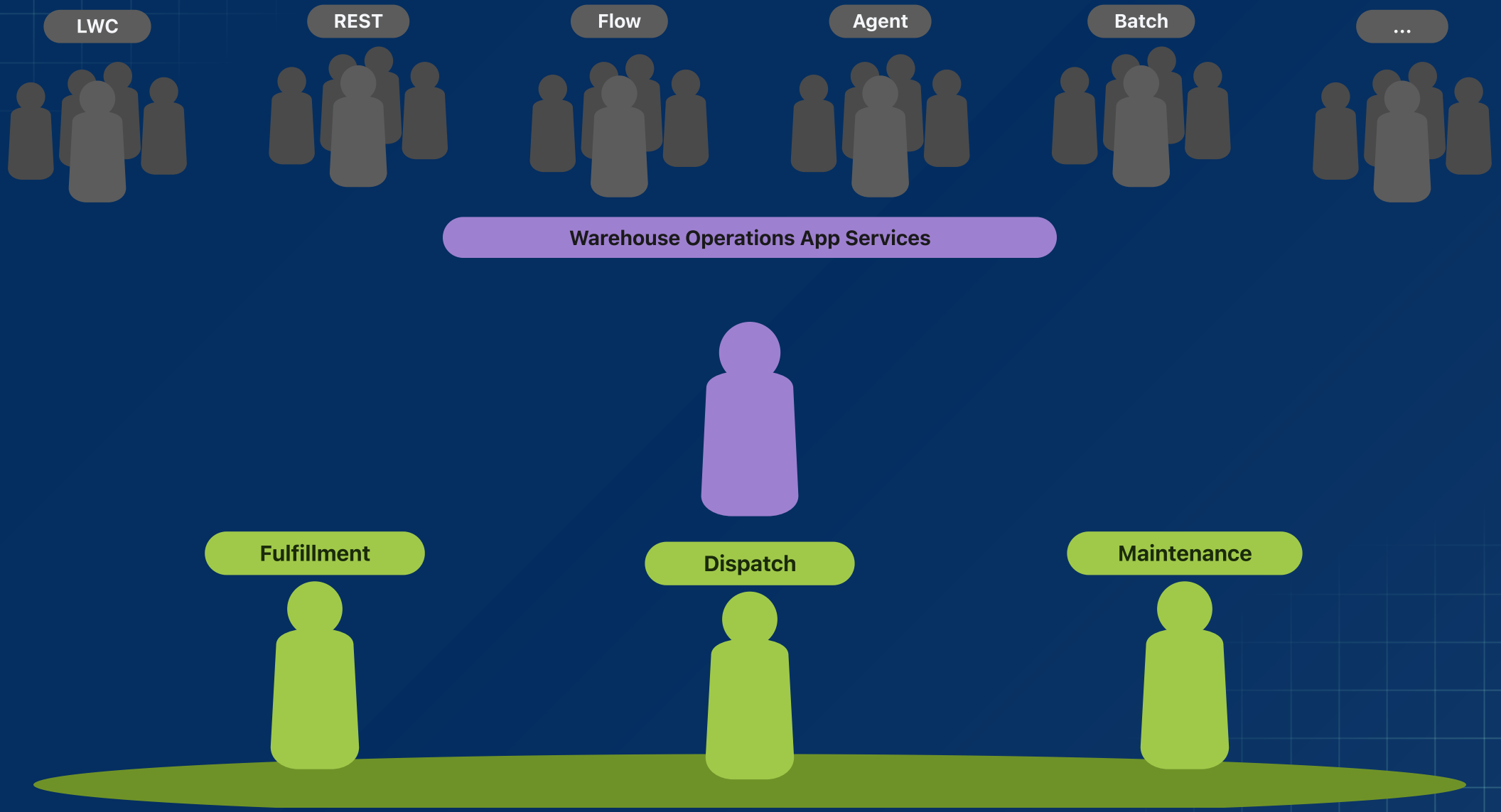
AndyInTheCloud Consulting Ltd



The series

	SESSION
✅	Session #1 - Separation of Concerns in Apex: Why Your Future Self Will Thank You
☐	Session #2 - Service Layers Explained: Coordinating Business Logic in Apex
☐	Session #3 - Domain vs Service: Where Should Your Apex Logic Live?
☐	Session #4 - Query Logic as a First-Class Architecture Concern
☐	Session #5 - Mocking in Apex: Why It Changes Everything
☐	Session #6 - Enterprise-Scale Apex Across Multiple Packages

Coordinating Business Logic in Apex

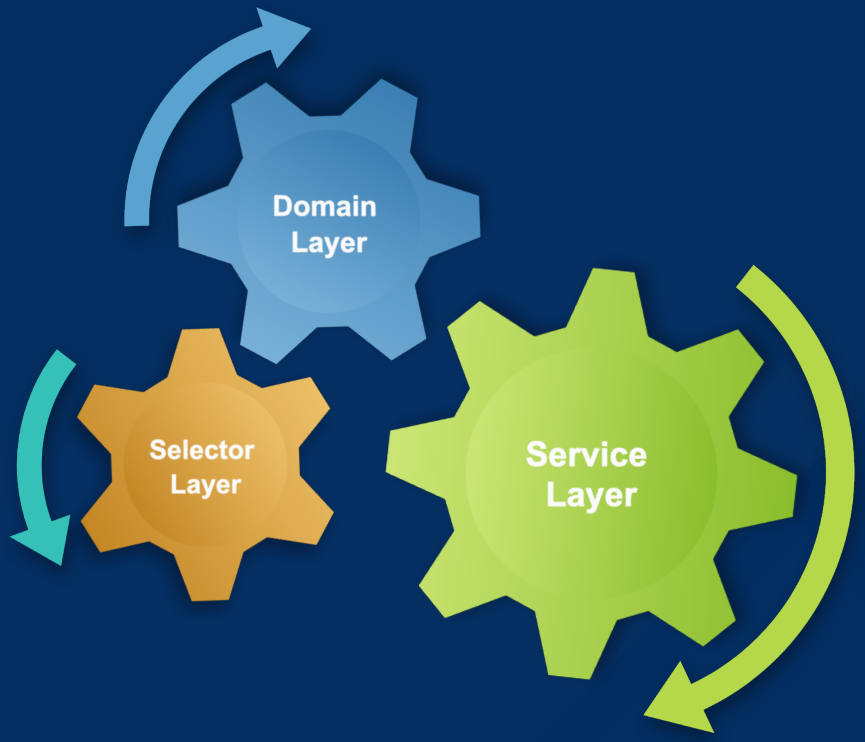


Service Layer

> *Defines an application's boundary with a layer of services that establishes a set of available operations and coordinates the application's response in each operation.*

Martin Fowler — *Patterns of Enterprise Application Architecture*

Service Layers Explained



SERVICE LAYERS EXPLAINED

- [x] Separation of Concerns Recap
- [ ] The Unit of Work
- [ ] Service Layer Principles
- [ ] Warehouse Operations App Overview
- [ ] Warehouse Operations App Code
- [ ] Warehouse Operations Agent

Salesforce Platform Layers

Presentation

- **You define:** Layouts, Custom Buttons, Flow, Record Types, Formula, Reports, Dashboards, Agent Builder
- **You code:** Apex Controllers, Visualforce, Aura, Lightning Components, React

Integration

- **You define:** Custom Objects, Custom Fields
- **You code:** Apex SOAP, Apex REST, Invocable Methods, Agent Actions, MCP, `global` Apex

Business Logic

- **You define:** Formula, Validation Rules, Workflow, Process Builder, Flow, Sharing Rules, Prompt Builder
- **You code:** Apex Services, DataWeave, Callouts

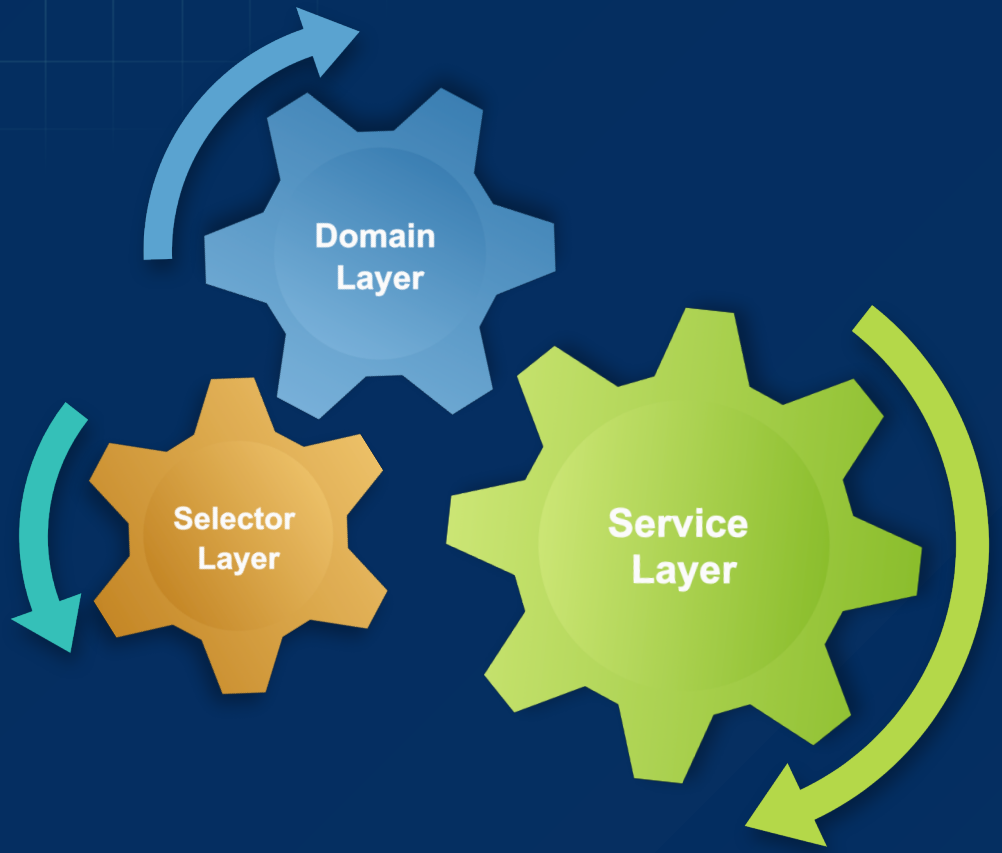
Data Access

- **You define:** Data Loaders, Permission Sets, Profiles
- **You code:** SOQL, SOSL and Salesforce APIs

Database

- **You define:** Custom Objects, Custom Fields, Relationships, Rollups, D360
- **You code:** Apex Triggers, External Data Sources (External Objects)

Apex Enterprise Patterns (fflib) - Color Chart



- Client
- Service
- Domain
- Trigger Handler
- Selector

Suggested Class Folder Layout

Presentation

FulfillmentReleaseController.cls | WarehouseDispatchController.cls | MaintenanceCompleteController.cls | .

```
// /classes/controllers
```

Integration

ReleaseOrders.cls | DispatchWarehouse.cls | FulfillmentResource.cls

```
// /classes/actions · /classes/restapis
```

❤️ **Business Logic**

FulfillmentService.cls | DispatchService.cls | MaintenanceService.cls

```
// /classes/services — named for the process
```

Robots.cls | FulfillmentOrders.cls | FulfillmentLines.cls | ...

```
// /classes/domains — named for the object
```

Data Access

RobotsSelector.cls | FulfillmentLinesSelector.cls | WarehousesSelector.cls

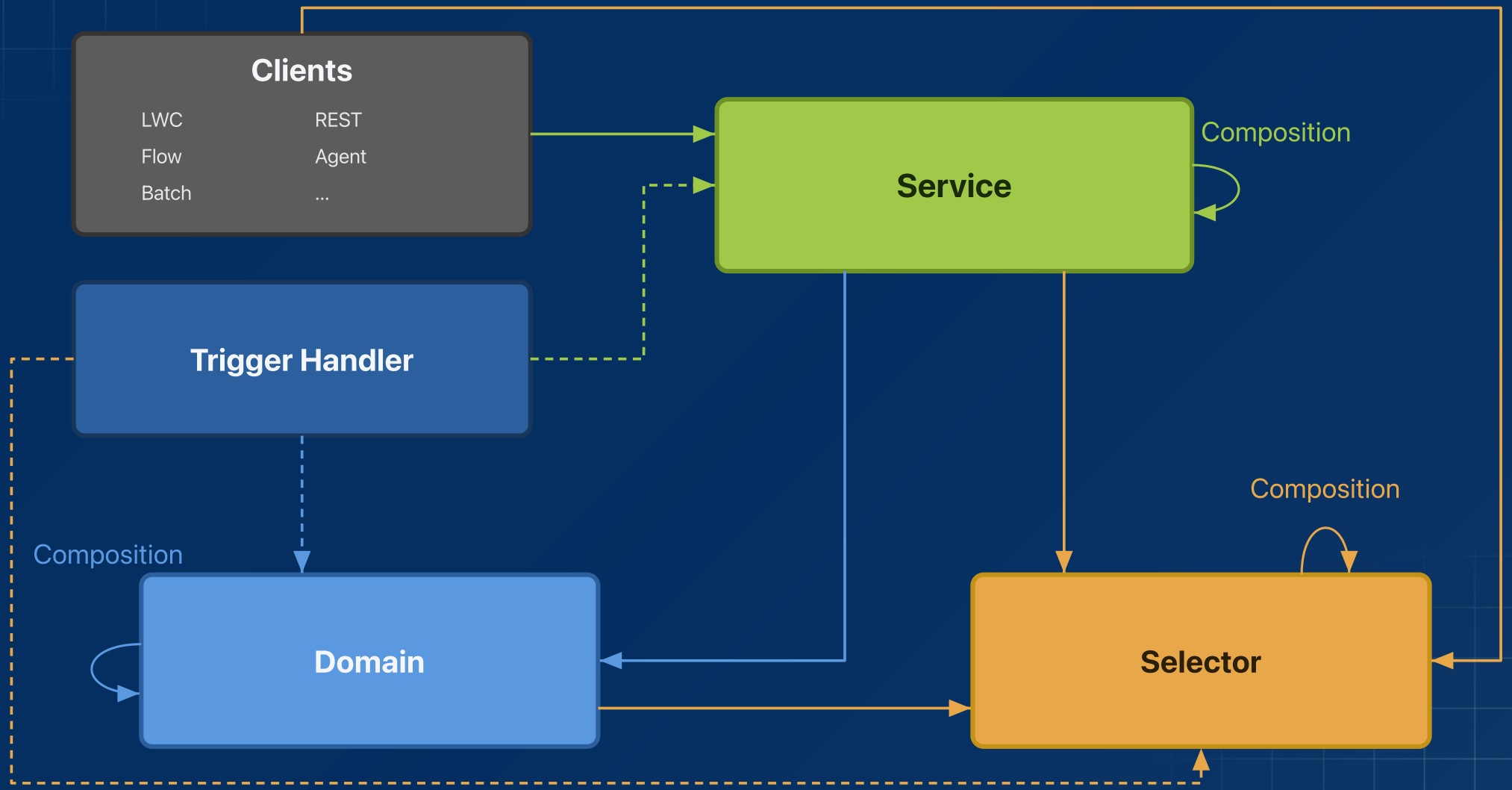
```
// /classes/selectors
```

Database

RobotsTriggerHandler.cls

```
// /classes/triggerHandlers
```

Who can call whom



Who can call whom

CALLER	Service	Domain	Selector
Client (LWC, REST, Flow, Agent, Batch, ...)	•		•
Trigger Handler	•	•	•
Service	•	•	•
Domain		•	•
Selector			•

How durable is your code when change occurs?

How users interact with your application changes a lot — protect against it with SoC

VF 🤔 Aura 🤔 LWC 🤔 Agent Action 🤔 React 🤔 Next?

🤔 Business Logic Impact

WarehouseDispatchController.cls | DispatchWarehouse.cls | FulfillmentReleaseController.cls | ReleaseOrders.cls

```
// logic copied — every new client type
VF · Aura · LWC · Agent Action · React · Next?
  ↳ WarehouseDispatchController.dispatchWarehouse(...)
  ↳ DispatchWarehouse.execute(...)
  ↳ FulfillmentReleaseController.releaseOrder(...)
  ↳ ReleaseOrders.execute(...)
```

ⓘ **Claudeforce. Salesforce + Anthropic, August 2026 — yet another interaction layer on your application.**

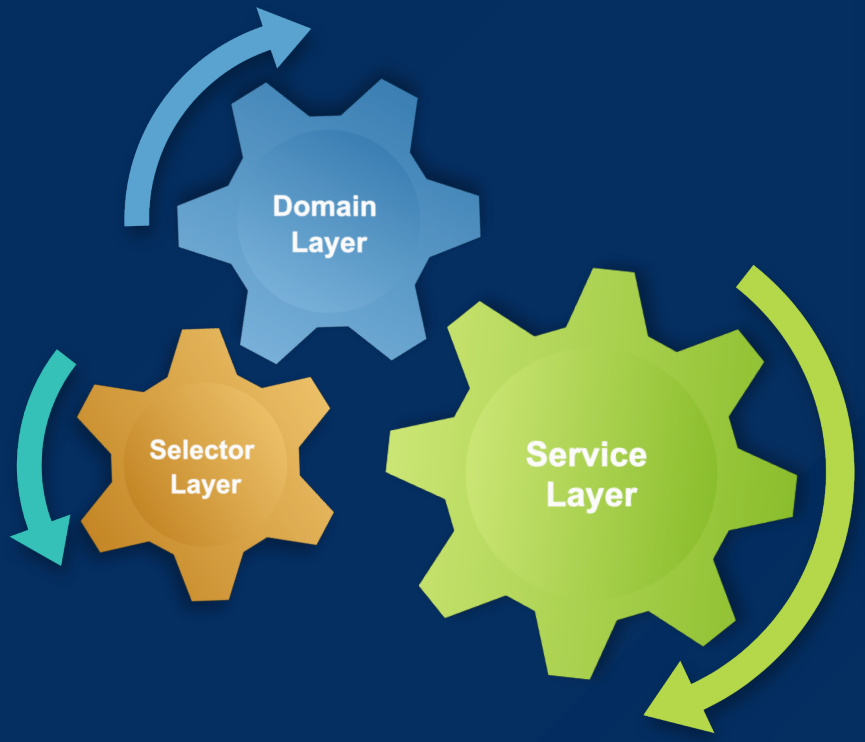
VF ❤️ Aura ❤️ LWC ❤️ Agent Action ❤️ React ❤️ Next?

DispatchService.cls | FulfillmentService.cls

```
// same services — every client type
LWC · REST · Batch · Flow · Agent Action · Next?
  ↳ DispatchService.dispatchWarehouses(...)
  ↳ FulfillmentService.releaseOrders(...)
```

❤️ Your business logic is **the** most important logic in your app — and when it's everywhere, it becomes a **significant investment risk**

Service Layers Explained



SERVICE LAYERS EXPLAINED	
Separation of Concerns Recap	✅
The Unit of Work	☐
Service Layer Principles	☐
Warehouse Operations App Overview	☐
Warehouse Operations App Code	☐
Warehouse Operations Agent	☐

Unit of Work

- Martin Fowler — *Patterns of Enterprise Application Architecture*
  - *"Maintains a list of objects affected by a business transaction and coordinates the writing out of changes and the resolution of concurrency problems."*
- One transaction: work completes or is rolled back explicitly
- Best practices: automatically bulkifies, runs in user mode
- Relationships: built in memory without having to insert parent
- `commitWork()` takes a savepoint and rolls back if anything fails
- In Apex that means: register work now, `commitWork()` once — see AndyInTheCloud, June 2013

Without Unit Of Work — Lines: 73

Opportunity setup — raw DML

```
1  List<Opportunity> opps = new List<Opportunity>();
2  List<List<Product2>> productsByOpp = new List<List<Product2>>();
3  List<List<PricebookEntry>> pbesByOpp = new List<List<PricebookEntry>>();
4  List<List<OpportunityLineItem>> linesByOpp = new List<List<OpportunityLineItem>>();
5  for (Integer o = 0; o < 10; o++) {
7    Opportunity opp = new Opportunity();
8    // ... set Name, StageName, CloseDate
11   opps.add(opp);
12   List<Product2> products = new List<Product2>();
13   List<PricebookEntry> pbes = new List<PricebookEntry>();
14   List<OpportunityLineItem> lines = new List<OpportunityLineItem>();
15   for (Integer i = 0; i < o + 1; i++) {
17     Product2 product = new Product2();
18     // ... set Name
19     products.add(product);
20     PricebookEntry pbe = new PricebookEntry();
21     // ... set UnitPrice, IsActive, Pricebook2Id
25     pbes.add(pbe);
26     OpportunityLineItem oli = new OpportunityLineItem();
27     // ... set Quantity, TotalPrice
29     lines.add(oli);
30   }
31   productsByOpp.add(products);
32   pbesByOpp.add(pbes);
33   linesByOpp.add(lines);
34 }
36 insert opps;
38 List<Product2> allProducts = new List<Product2>();
39 for (List<Product2> products : productsByOpp) {
41   allProducts.addAll(products);
42 }
43 insert allProducts;
45 Integer oppIdx = 0;
46 List<PricebookEntry> allPbes = new List<PricebookEntry>();
47 for (List<PricebookEntry> pbes : pbesByOpp) {
49   List<Product2> products = productsByOpp[oppIdx++];
50   Integer lineIdx = 0;
51   for (PricebookEntry pbe : pbes) {
53     pbe.Product2Id = products[lineIdx++].Id;
54   }
55   allPbes.addAll(pbes);
56 }
57 insert allPbes;
59 oppIdx = 0;
60 List<OpportunityLineItem> allLines = new List<OpportunityLineItem>();
61 for (List<OpportunityLineItem> lines : linesByOpp) {
63   List<PricebookEntry> pbes = pbesByOpp[oppIdx];
64   Integer lineIdx = 0;
65   for (OpportunityLineItem oli : lines) {
67     oli.OpportunityId = opps[oppIdx].Id;
68     oli.PricebookEntryId = pbes[lineIdx++].Id;
69   }
70   allLines.addAll(lines);
71   oppIdx++;
72 }
73 insert allLines;
```

Zoom · lines 1–4

```
1 List<Opportunity> opps = new List<Opportunity>();
2 List<List<Product2>> productsByOpp = new List<List<Product2>>();
3 List<List<PricebookEntry>> pbesByOpp = new List<List<PricebookEntry>>();
4 List<List<OpportunityLineItem>> linesByOpp = new List<List<OpportunityLineItem>>(
```

Zoom · lines 47–56

```
47 for (List<PricebookEntry> pbes : pbesByOpp) {
49   List<Product2> products = productsByOpp[oppIdx++];
50   Integer lineIdx = 0;
51   for (PricebookEntry pbe : pbes) {
53     pbe.Product2Id = products[lineIdx++].Id;
54   }
55   allPbes.addAll(pbes);
56 }
```

Zoom · lines 36, 43, 57, 73

```
36 insert opps;
43 insert allProducts;
57 insert allPbes;
73 insert allLines;
```

Zoom · Transaction control

```
1 Savepoint sp = Database.setSavepoint();
2 try {
3   // ... lists, stitch, four inserts
4 } catch (Exception e) {
5   Database.rollback(sp);
6   throw new DispatchServiceException(e.getMessage(), e);
7 }
```

With Unit of Work — Lines: 31 (58% ↓)

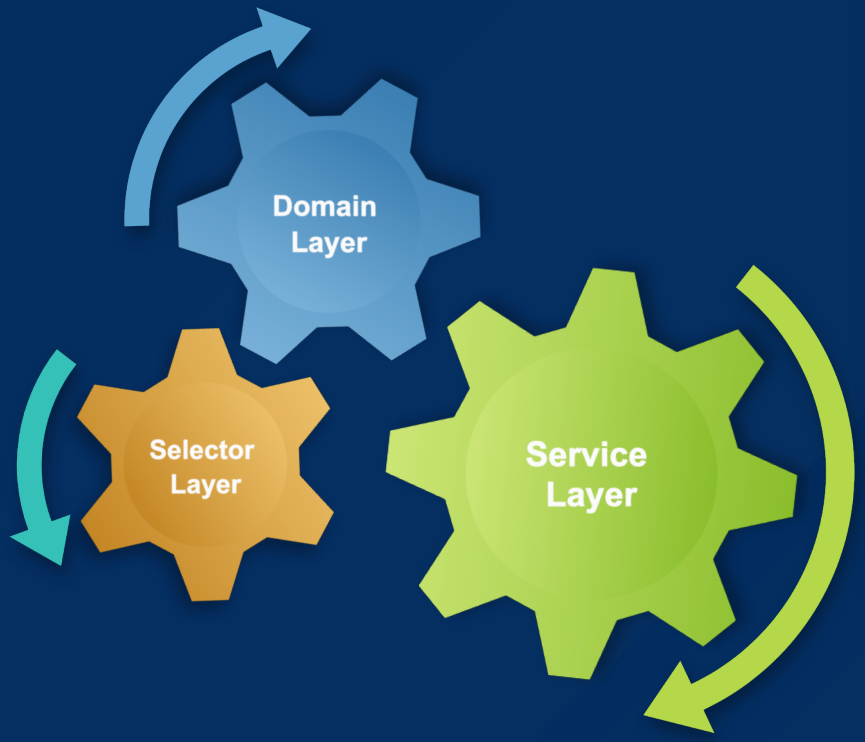
Same work — register, then commit

```
fflib_SObjectUnitOfWork uow = new fflib_SObjectUnitOfWork(
  new List<Schema.SObjectType> {
    Product2.SObjectType,
    PricebookEntry.SObjectType,
    Opportunity.SObjectType,
    OpportunityLineItem.SObjectType
  });
for (Integer o = 0; o < 10; o++) {
  Opportunity opp = new Opportunity();
  // ... set Name, StageName, CloseDate
  uow.registerNew(opp);
  for (Integer i = 0; i < o + 1; i++) {
    Product2 product = new Product2();
    product.Name = opp.Name + ' : Product : ' + i;
    uow.registerNew(product);
    PricebookEntry pbe = new PricebookEntry();
    // ... set UnitPrice, IsActive, Pricebook2Id
    uow.registerNew(pbe, PricebookEntry.Product2Id, product);
    OpportunityLineItem oli = new OpportunityLineItem();
    // ... set Quantity, TotalPrice
    uow.registerRelationship(oli, OpportunityLineItem.PricebookEntryId, pbe);
    uow.registerNew(oli, OpportunityLineItem.OpportunityId, opp);
  }
}
uow.commitWork();
```

fflib_SObjectUnitOfWork Methods

- `fflib_SObjectUnitOfWork(List<SObjectType> types)` — dependency order
- `registerNew(record)` · `registerNew(record, field, parent)`
- `registerRelationship(record, field, related)` — stitch before Ids exist
- `registerDirty(record)` · `registerDeleted(record)`
- `commitWork()` — savepoint, bulk DML, rollback, rethrow
- Other methods: `registerWork`, `registerUpsert`, `registerEmail` …

Service Layers Explained

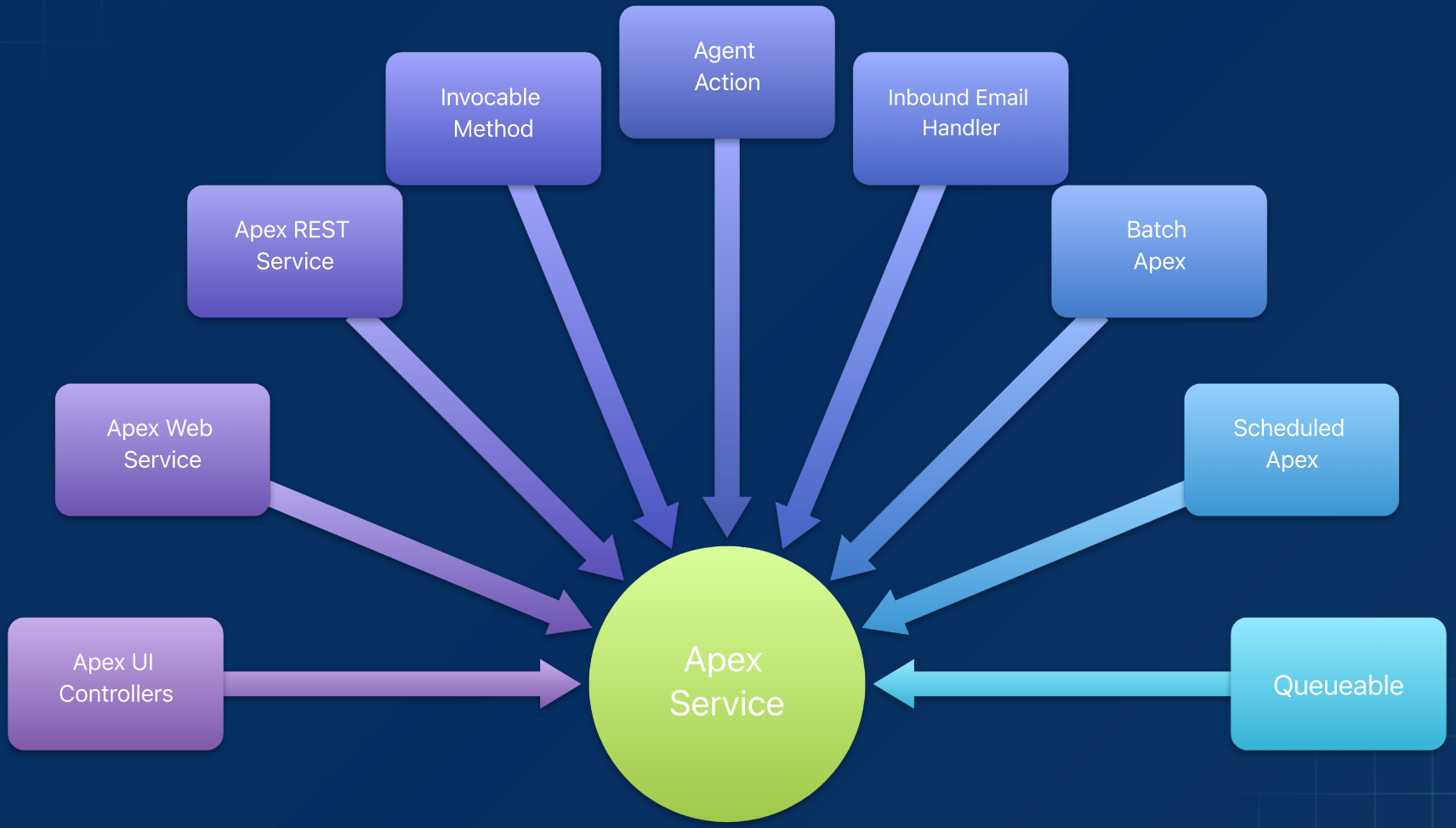


SERVICE LAYERS EXPLAINED	
Separation of Concerns Recap	✅
The Unit of Work	✅
Service Layer Principles	☐
Warehouse Operations App Overview	☐
Warehouse Operations App Code	☐
Warehouse Operations Agent	☐

Service ➡️ Task Orientated Logic

- A Service represents a **feature** of the application
  - Fulfillment — `FulfillmentService.cls`
  - Dispatch — `DispatchService.cls`
  - Maintenance — `MaintenanceService.cls`
- Each method is a **task** within that feature — `releaseOrders`, `dispatchWarehouses`, `completeLines`
- Think of it as the **conductor** — the band handles querying and object-specific logic; the Service sets the tempo and the rules of the orchestration

Services have many Consumers



Service ☑ Checklist

- Client-agnostic inputs and outputs
  - ✅ `dispatchWarehouses(Set<Id> warehouseIds)`
  - ❌ `dispatch(DispatchForm form)`
- Client-agnostic exceptions
  - ✅ `throw DispatchServiceException`
  - ❌ `throw AuraHandledException`
- **Bulkification** — collections in and out; don't force callers to loop
  - ✅ `dispatchWarehouses(Set<Id> warehouseIds)`
  - ❌ `dispatchWarehouse(Id warehouseId)` only
- **Transactions** — one UOW; rollback; Service→Service passes the outer uow
- **Security** — enforce user-mode in DML (UOW) and SOQL (Selector)

Service 🔁 Evolution

- **Services** — now prefer instance methods over static
  - ✅ `DispatchService.newInstance().dispatchWarehouses(...)`
  - ❌ `public static void dispatchWarehouses(...)`
- **Apex interfaces** — now recommended only for dependency injection
  - ✅ `IDispatchService` only when a test or package must inject
  - ❌ `IFulfillmentService` / `IDispatchService` / `IMaintenanceService` on every service by default
- **Application class** — optional; mocking can be done without it — blog

Service - Consumers and Dependencies

Service - Leveraging Unit of Work

Service.cls

Service Method Entry ...

```
public static void doSomeStuff (...) {
```

Create a Unit of Work

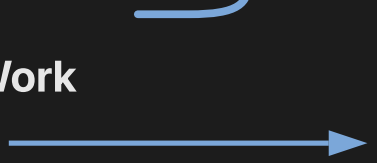
```
uow = new fflib_SObjectUnitOfWork
```

Do work, register record changes

```
uow.registerDirty
uow.registerNew
uow.registerDelete
```

Commit a Unit of Work

```
uow.commitWork
```



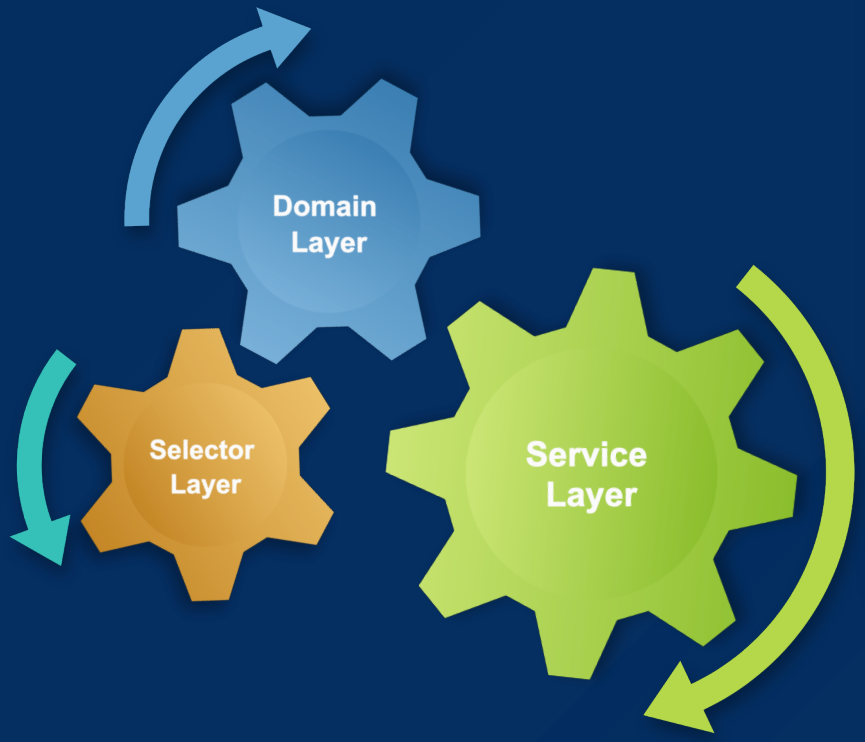
- Bulkified
- Secure by default

```
}
```

... Service Method Exit

ⓘ **If calling between services. Pass the outer Unit of Work as a parameter. Do not create a new one. Aim for one Unit of Work per request.**

Service Layers Explained

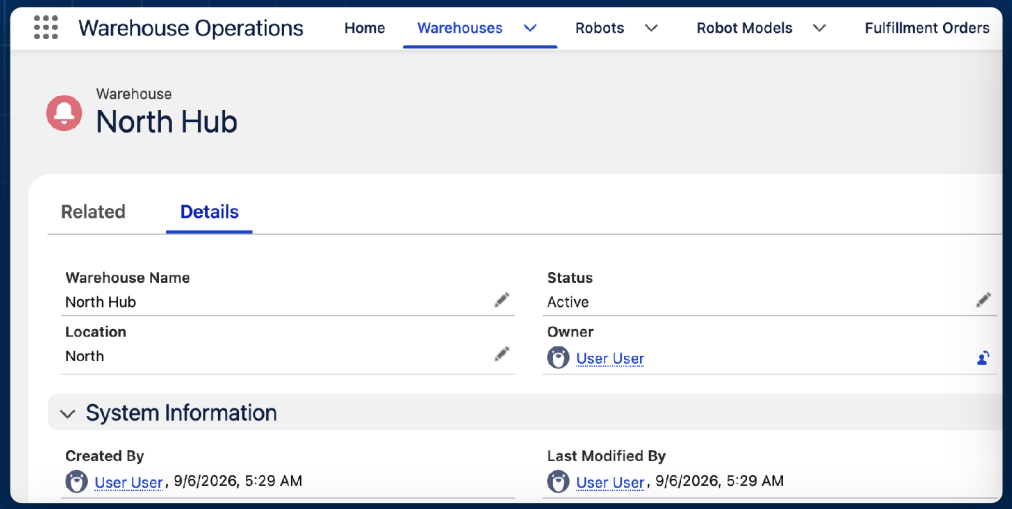


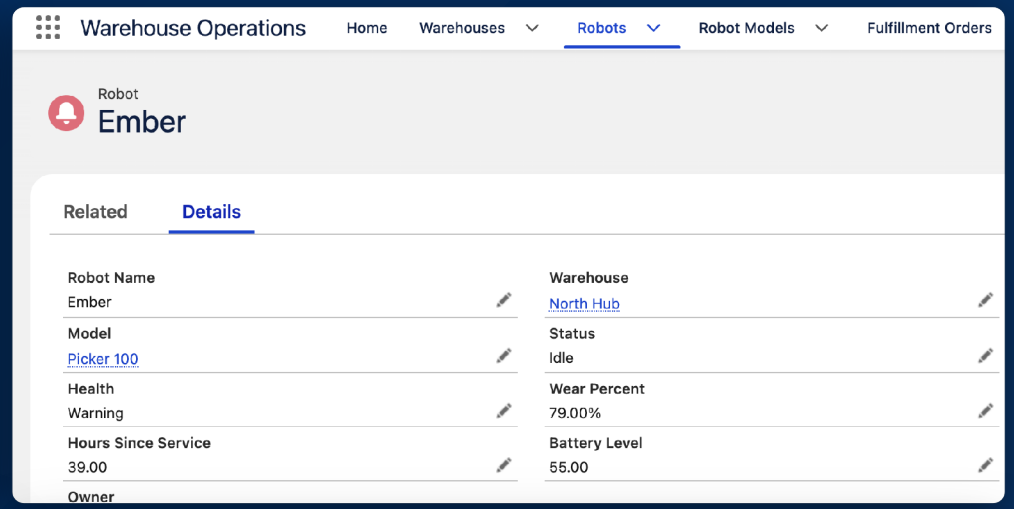
SERVICE LAYERS EXPLAINED	
Separation of Concerns Recap	✅
The Unit of Work	✅
Service Layer Principles	✅
Warehouse Operations App Overview	☐
Warehouse Operations App Code	☐
Warehouse Operations Agent	☐

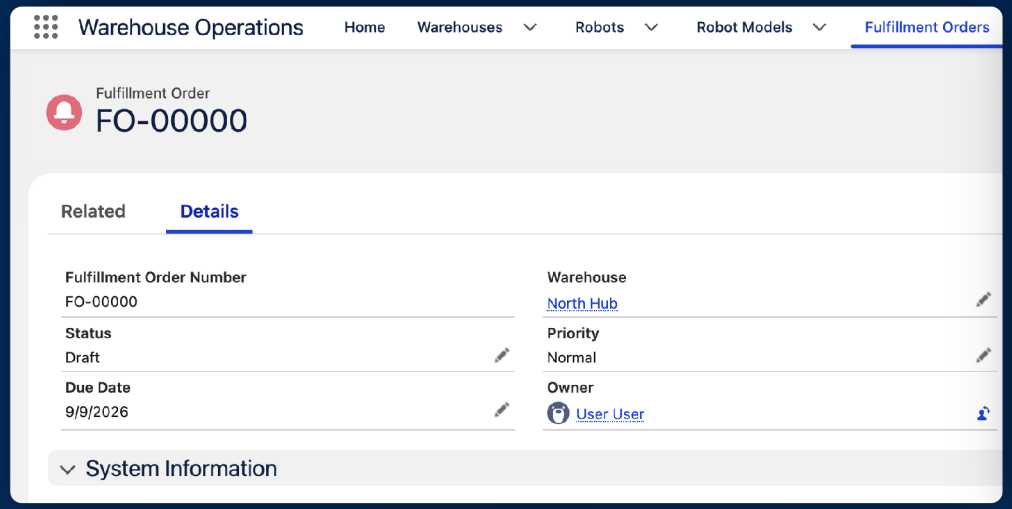
Warehouse Operations App

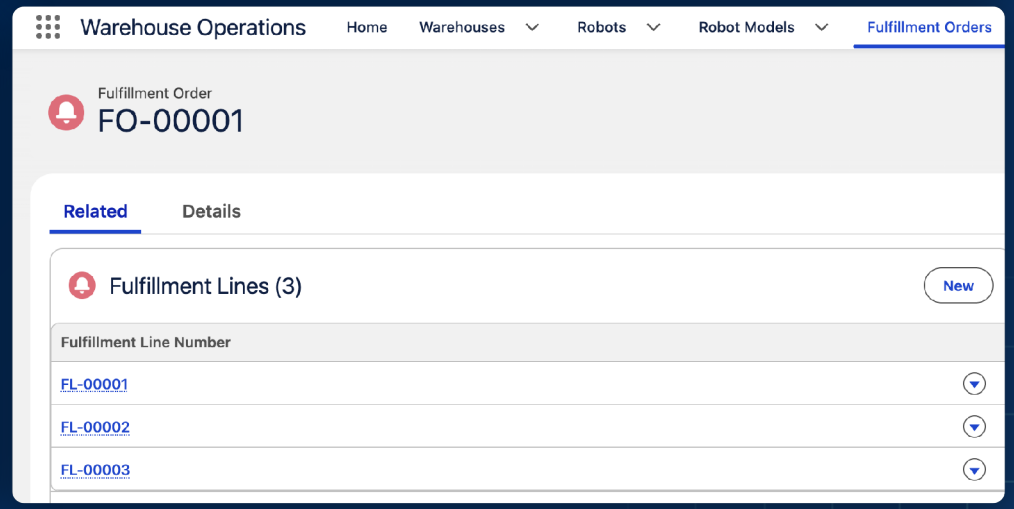
- The operations app for a warehouse that uses robots
- Release pick work, dispatch idle robots, take a worn one into maintenance
- The sample is the **desk**, not the robot brain
- Improve sample clarity between service vs domain
  - `DispatchService` — a process
  - `Robots` — an object
- Leverages multiple clients, LWC, REST API, Batch and AI

Warehouse App Tabs

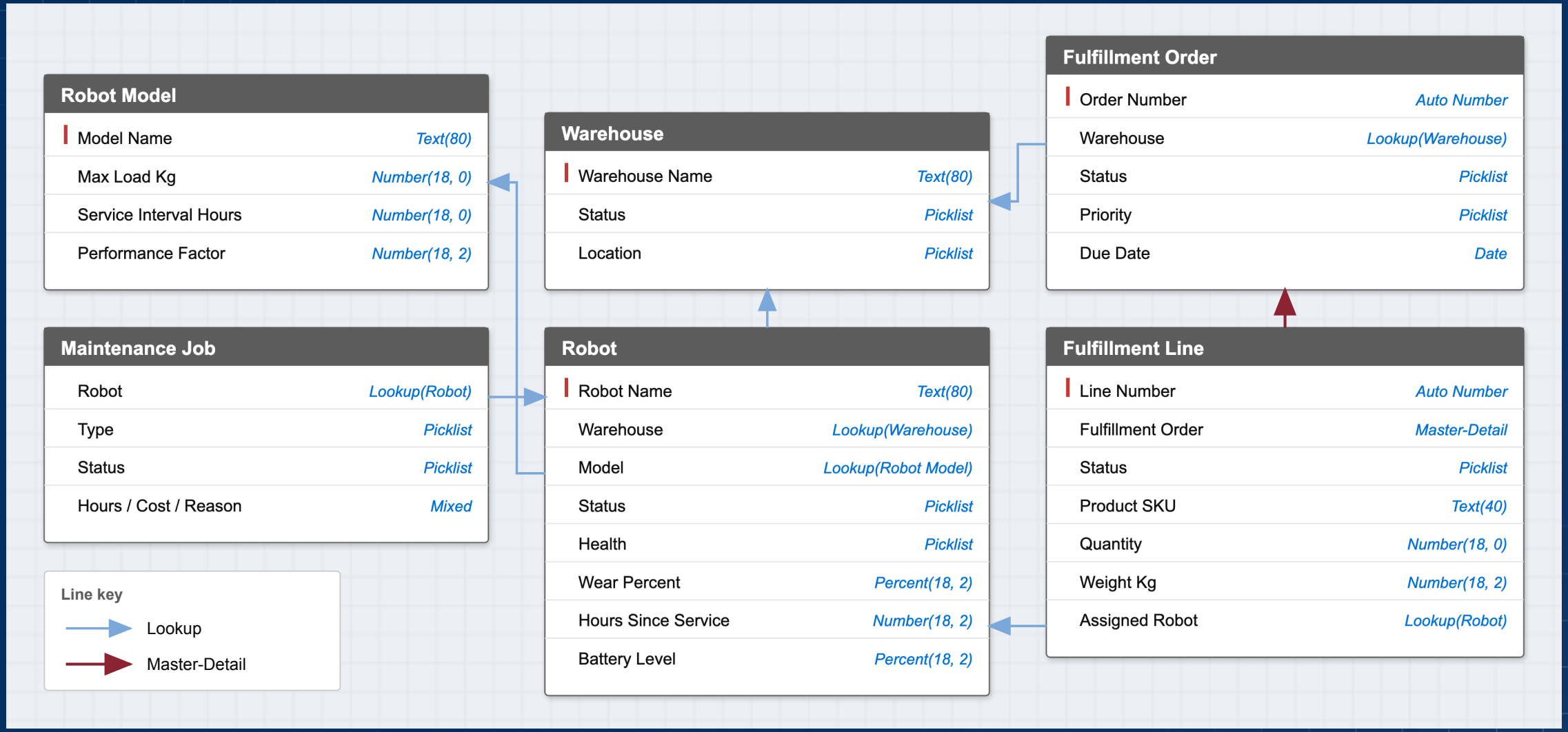






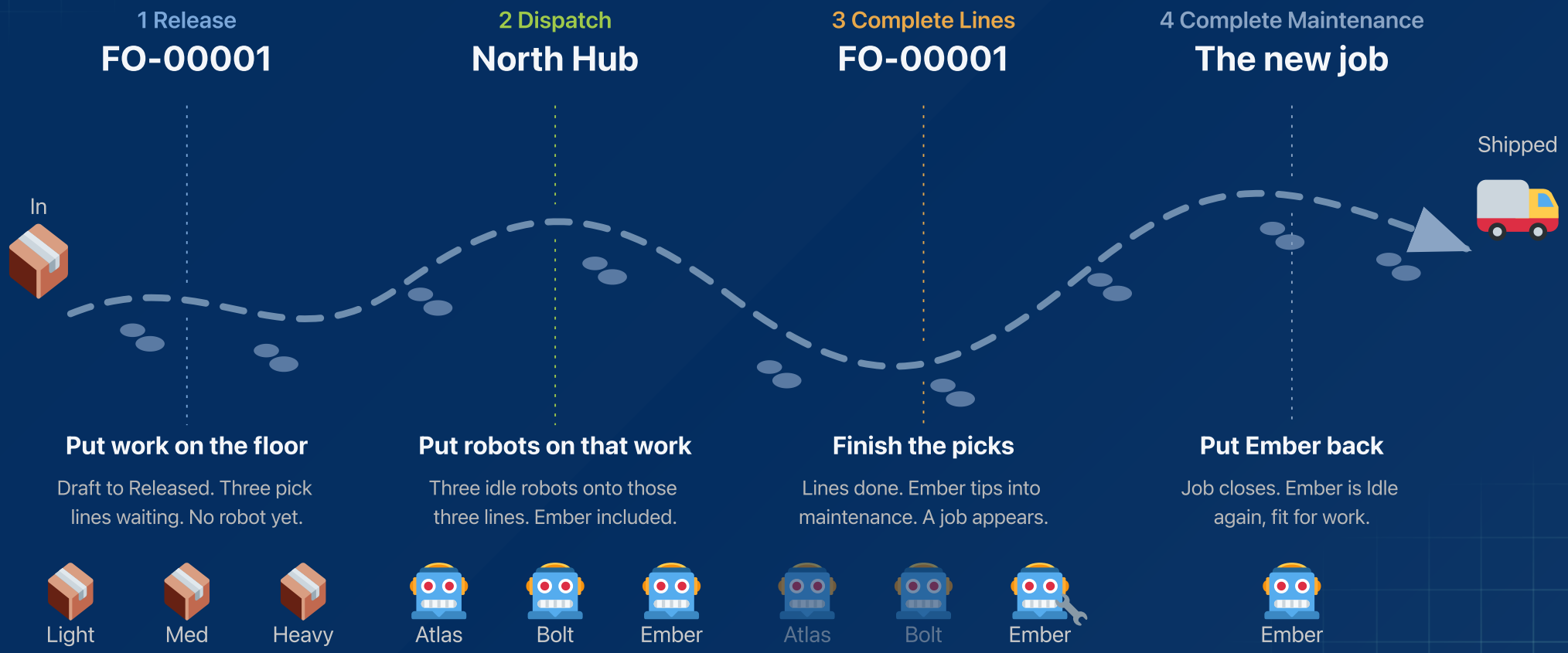


Warehouse App Objects

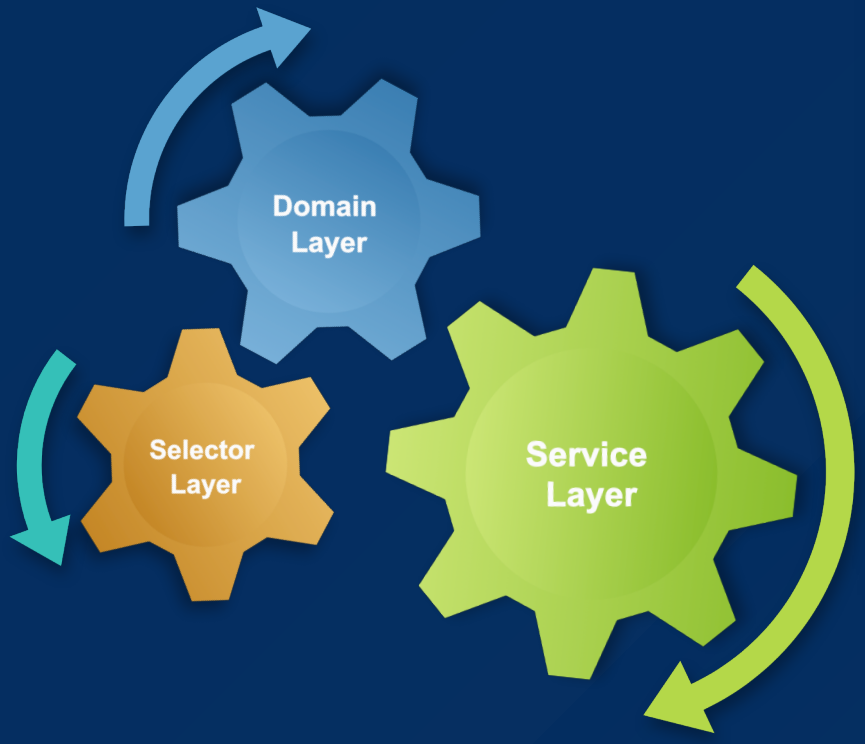


Warehouse App Processes

North Hub Warehouse Fulfillment

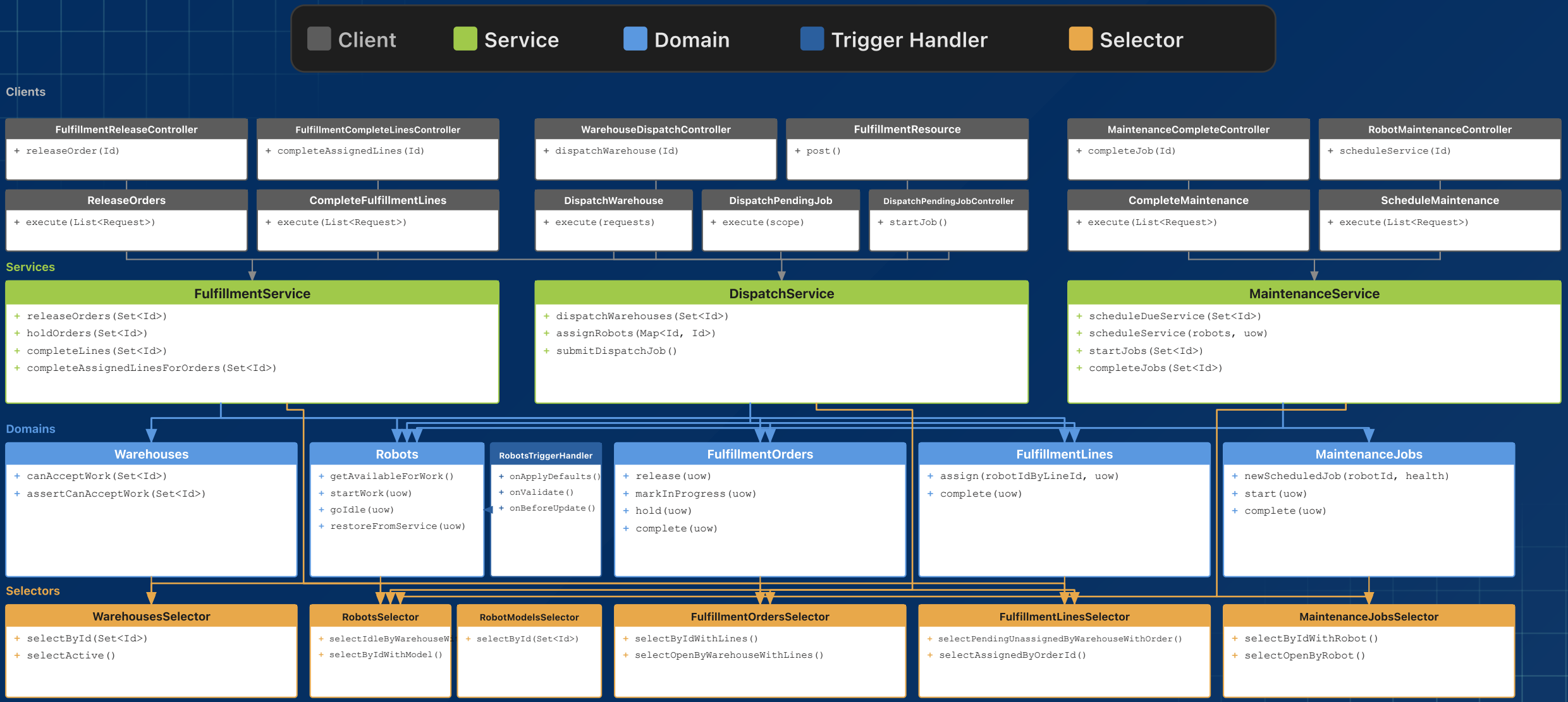


Service Layers Explained



SERVICE LAYERS EXPLAINED	
Separation of Concerns Recap	✅
The Unit of Work	✅
Service Layer Principles	✅
Warehouse Operations App Overview	✅
Warehouse Operations App Code	☐
Warehouse Operations Agent	☐

Warehouse App Classes



Client Service Domain Trigger Handler Selector

North Hub Warehouse Fulfillment

1 Release
FO-00001
Put work on the floor

FulfillmentReleaseController.cls
```
@AuraEnabled
releaseOrder (Id)
```
FulfillmentService.cls
```
releaseOrders ( Set<Id> )
```
FulfillmentOrders.cls
```
release (uow)
```
FulfillmentOrdersSelector.cls
```
selectByIdWithLines ( Set<Id> )
```

2 Dispatch
North Hub
Put robots on that work

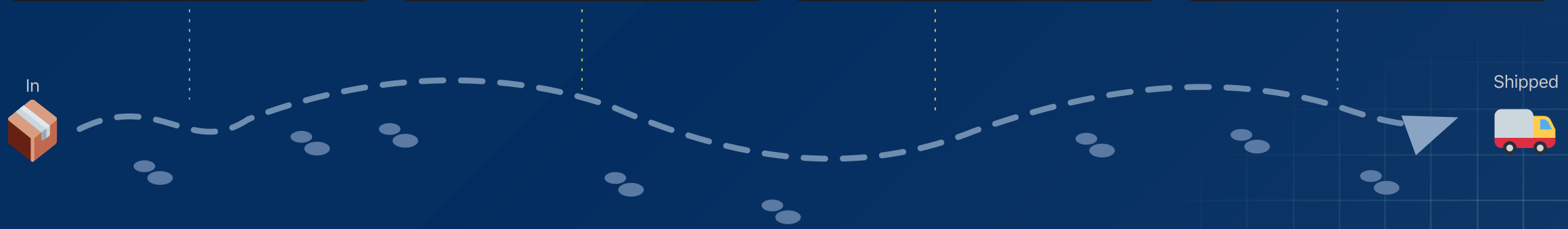
WarehouseDispatchController.cls
```
@AuraEnabled
dispatchWarehouse (Id)
```
DispatchService.cls
```
dispatchWarehouses ( Set<Id> )
```
Robots.cls
```
getAvailableForWork ()
```
RobotsSelector.cls
```
selectIdleByWarehouseWithModel ( Set<Id> )
```

3 Complete Lines
FO-00001
Finish the picks

FulfillmentCompleteLinesController.cls
```
@AuraEnabled
completeAssignedLines (Id)
```
FulfillmentService.cls
```
completeAssignedLinesForOrders ( Set<Id> )
```
MaintenanceService.cls
```
scheduleService (robots, uow)
```
Robots.cls
```
goIdle (uow) applyWear (uow)
```

4 Complete Maintenance
The new job
Put Ember back

MaintenanceCompleteController.cls
```
@AuraEnabled
completeJob (Id)
```
MaintenanceService.cls
```
completeJobs ( Set<Id> )
```
MaintenanceJobs.cls
```
complete (uow)
```
Robots.cls
```
restoreFromService (uow)
```



WarehouseDispatchController.cls

```
1  public inherited sharing class WarehouseDispatchController {
2    @AuraEnabled(cacheable=false)
3    public static void dispatchWarehouse(Id warehouseId) {
4      try {
5        DispatchService.newInstance()
6            .dispatchWarehouses(new Set<Id>{ warehouseId });
7      } catch (Exception e) {
8        throw toAura(e);
9      }
10   }
11
12   private static AuraHandledException toAura(Exception e) {
13     AuraHandledException auraException = new AuraHandledException(e.getMessage
14     auraException.setMessage(e.getMessage());
15     return auraException;
16   }
17 }
```

DispatchPendingJob.cls

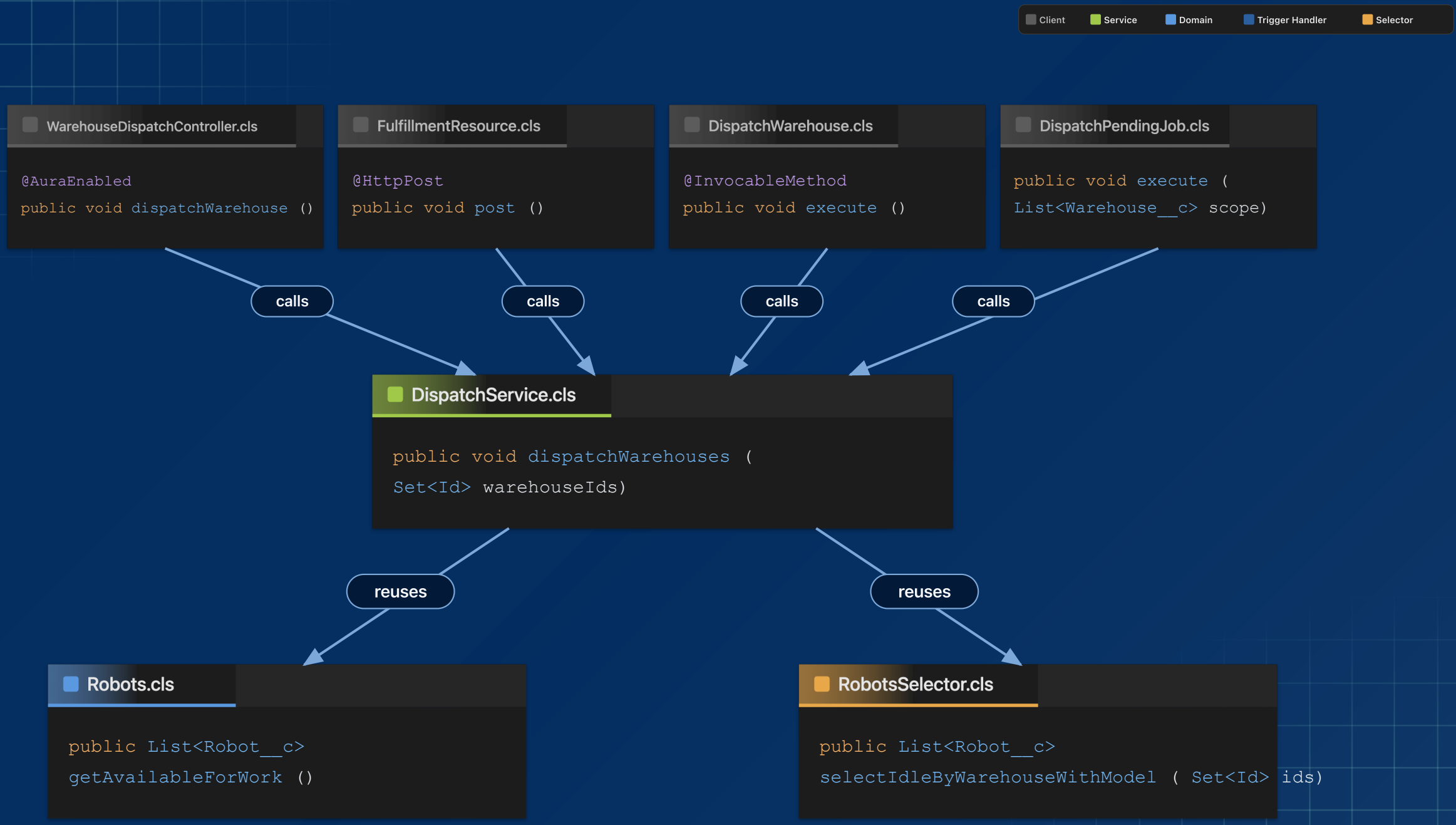
```
1  public inherited sharing class DispatchPendingJob
2      implements System.Schedulable, Database.Batchable<SObject>, Database.State
3
4    public Database.QueryLocator start(Database.BatchableContext context) {
5      return WarehousesSelector.newInstance()
6          .selectActiveAsQueryLocator();
7    }
8
9    public void execute(
10       Database.BatchableContext context,
11       List<Warehouse__c> warehouses) {
12     try {
13       DispatchService.newInstance()
14           .dispatchWarehouses(new Map<Id, Warehouse__c>(warehouses).keySet());
15     } catch (Exception e) {
16       jobErrors.add(/* JobError */);
17     }
18   }
19
20   public void finish(Database.BatchableContext context) { /* email JobErrors */
21 }
```

FulfillmentResource.cls

```
1  @RestResource(UrlMapping='/warehouse/fulfillment/*')
2  global inherited sharing class FulfillmentResource {
3    @HttpPost
4    global static void post() {
5      Request body = (Request) JSON.deserialize(
6          RestContext.request.requestBody.toString(), Request.class);
7      String action = /* last URI segment */;
8      switch on action {
9        when 'release' {
10         FulfillmentService.newInstance()
11             .releaseOrders(new Set<Id>(body.orderIds));
12       }
13       when 'dispatch' {
14         DispatchService.newInstance()
15             .dispatchWarehouses(new Set<Id>(body.warehouseIds));
16       }
17       when 'complete' {
18         FulfillmentService.newInstance()
19             .completeLines(new Set<Id>(body.lineIds));
20       }
21     }
```

DispatchWarehouse.cls

```apex
public inherited sharing class DispatchWarehouse {
  @InvocableMethod(
    label='Dispatch Warehouse'
    category='Warehouse Operations')
  public static List<Result> execute(List<Request> requests) {
    Set<Id> warehouseIds = new Set<Id>();
    for (Request request : requests) {
      warehouseIds.add(request.warehouseId);
    }
    DispatchService.newInstance().dispatchWarehouses(warehouseIds);
    List<Result> results = new List<Result>();
    for (Request request : requests) {
      Result result = new Result();
      result.warehouseId = request.warehouseId;
      results.add(result);
    }
    return results;
  }
}
```


Client
Service
Domain
Trigger Handler
Selector
WarehouseDispatchController.cls
@AuraEnabled
public void dispatchWarehouse ()
FulfillmentResource.cls
@HttpPost
public void post ()
DispatchWarehouse.cls
@InvocableMethod
public void execute ()
DispatchPendingJob.cls
public void execute (
List<Warehouse__c> scope)
calls
calls
calls
calls
DispatchService.cls
public void dispatchWarehouses (
Set<Id> warehouseIds)
reuses
reuses
Robots.cls
public List<Robot__c>
getAvailableForWork ()
RobotsSelector.cls
public List<Robot__c>
selectIdleByWarehouseWithModel (Set<Id> ids)

DispatchService.dispatchWarehouses

```
public virtual void dispatchWarehouses(Set<Id> warehouseIds) {
```

1. **Service** — new `fflib_SObjectUnitOfWork`
2. **Selector** — pending unassigned lines at the warehouses
3. **Selector** — idle robots, with model max load
4. **Domain** — `Robots.getAvailableForWork()` (Idle, not Critical, wear < 90, battery > 20)
5. **Service** — `match` (per warehouse, line weight vs robot max load)
6. **Domain** — `Robots.startWork(uow)` · `FulfillmentLines.assign(uow)` · `FulfillmentOrders.markInProgress(uow)`
7. **Service** — `uow.commitWork()`

```
}
```

DispatchService.dispatchWarehouses

```
public virtual void dispatchWarehouses(Set<Id> warehouseIds) {
  fflib_SObjectUnitOfWork uow = UnitOfWork.newInstance();

  FulfillmentLinesSelector lineSelector =
      FulfillmentLinesSelector.newInstance();
  RobotsSelector robotSelector =
      RobotsSelector.newInstance();
  List<FulfillmentLine__c> pending =
      lineSelector.selectPendingUnassignedByWarehouseWithOrder(warehouseIds);
  List<Robot__c> idle =
      robotSelector.selectIdleByWarehouseWithModel(warehouseIds);

  List<Robot__c> available =
      Robots.newInstance(idle).getAvailableForWork();
  Map<Id, Id> robotIdByLineId = match(pending, available);
  Robots.newInstance(robotsToStart).startWork(uow);
  FulfillmentLines.newInstance(linesToAssign).assign(robotIdByLineId, uow);
  FulfillmentOrders.newInstance(released).markInProgress(uow);

  uow.commitWork();
}
```

UnitOfWork.cls

```
public static fflib_SObjectUnitOfWork newInstance() {
  return new fflib_SObjectUnitOfWork(
    new List<SObjectType>{
      Warehouse__c.SObjectType,
      RobotModel__c.SObjectType,
      Robot__c.SObjectType,
      FulfillmentOrder__c.SObjectType,
      FulfillmentLine__c.SObjectType,
      MaintenanceJob__c.SObjectType
    },
    new fflib_SObjectUnitOfWork.UserModeDML()
  );
}
```

FulfillmentService.completeLines

```
public virtual void completeLines(Set<Id> lineIds) {
```

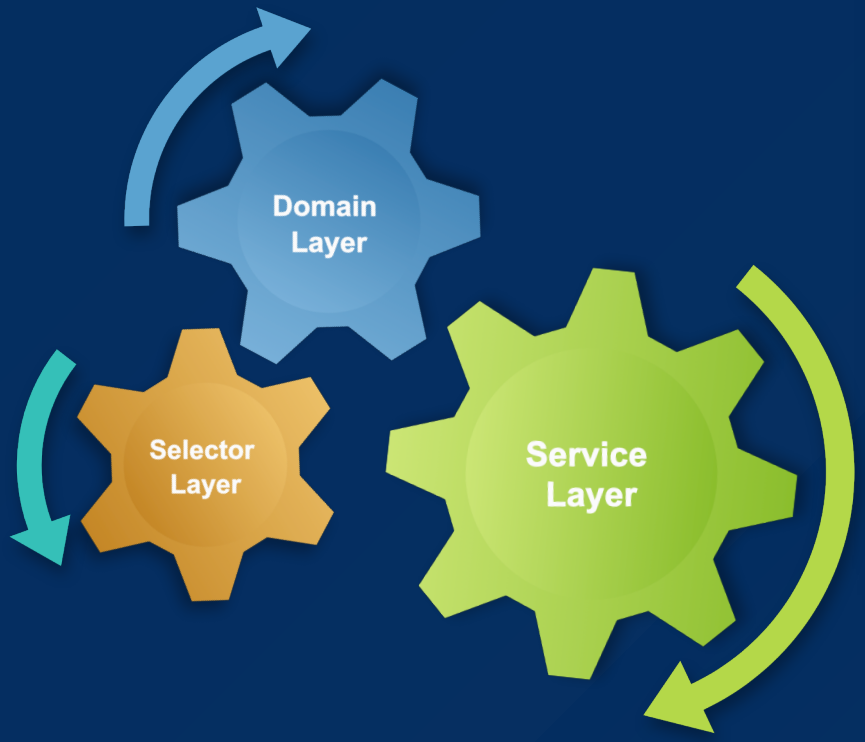
1. **Service** — new `fflib_SObjectUnitOfWork`
2. **Selector** — lines by Id (`lineIds`)
3. **Domain** — `FulfillmentLines.complete(uow)` · hours by assigned robot
4. **Selector** — robots with model (`hoursByRobotId`)
5. **Domain** — `Robots.goIdle(uow)` · `applyWear(uow)`
6. **Selector** — parent orders with every sibling line
7. **Domain** — `FulfillmentOrders.complete(uow, lineIds)` · `Robots.getDueForService()`
8. **Service** — `MaintenanceService.scheduleService(dueRobots, uow)`
9. **Service** — `uow.commitWork()`

```
}
```

FulfillmentService.completeLines

```
1  public virtual void completeLines(List<FulfillmentLine__c> lines) {
2    fflib_SObjectUnitOfWork uow = UnitOfWork.newInstance();
3
4    FulfillmentLines fulfillmentLines = FulfillmentLines.newInstance(lines);
5    fulfillmentLines.complete(uow);
6    Map<Id, Decimal> hoursByRobotId =
7        fulfillmentLines.getEstimatedHoursByAssignedRobotId();
8    List<Robot__c> robotRecords =
9        robotSelector.selectByIdWithModel(hoursByRobotId.keySet());
10   Robots robots = Robots.newInstance(robotRecords);
11   robots.goIdle(uow); robots.applyWear(hoursByRobotId, uow);
12
13   List<FulfillmentOrder__c> orders = orderSelector.selectByIdWithLines(orderIds
14   FulfillmentOrders.newInstance(orders).complete(uow, lineIds);
15
16   Set<Robot__c> dueRobots = robots.getDueForService();
17   if (!dueRobots.isEmpty()) {
18     MaintenanceService.newInstance().scheduleService(dueRobots, uow); // same
19   }
20   uow.commitWork();
21 }
```

Service Layers Explained



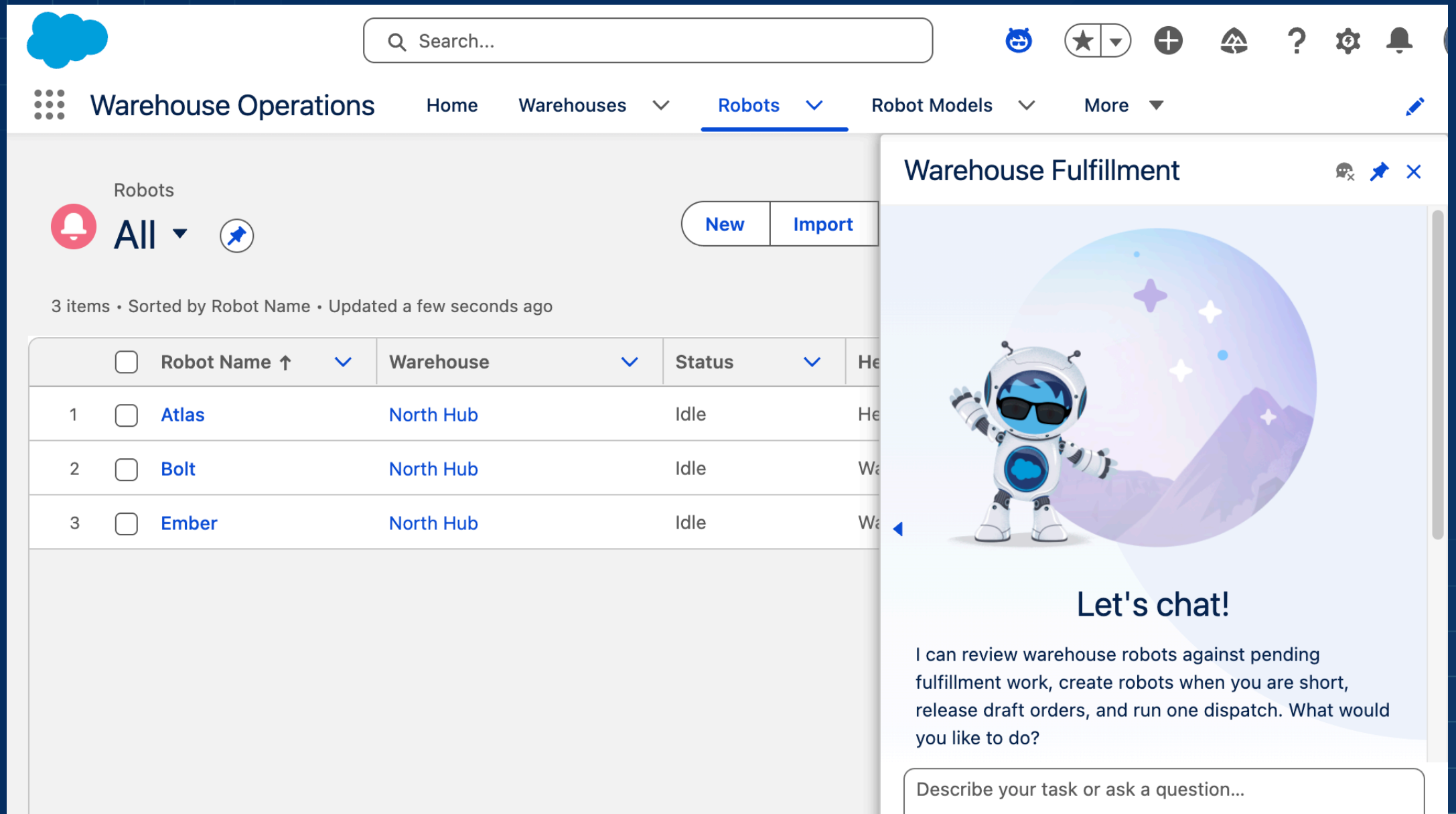
SERVICE LAYERS EXPLAINED	
Separation of Concerns Recap	✅
The Unit of Work	✅
Service Layer Principles	✅
Warehouse Operations App Overview	✅
Warehouse Operations App Code	✅
Warehouse Operations Agent	⬜

Warehouse Agent Actions

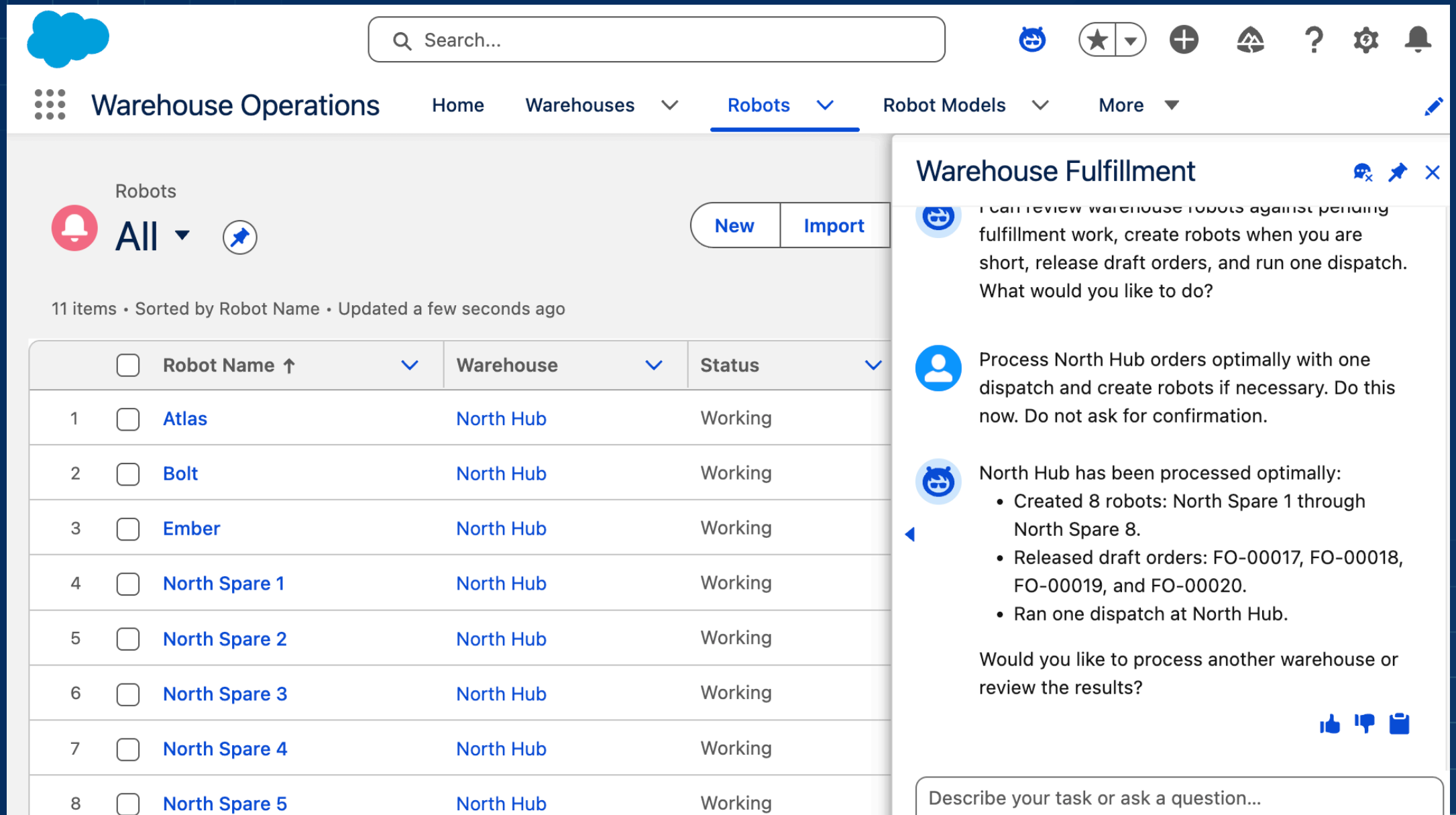
Warehouse Fulfillment

Actions	Services
GetWarehouseDesk.cls	WarehouseService.cls
CreateRobot (Flow)	
ReleaseOrders.cls	FulfillmentService.cls
DispatchWarehouse.cls	DispatchService.cls
CompleteFulfillmentLines.cls	FulfillmentService.cls
CompleteMaintenance.cls	MaintenanceService.cls

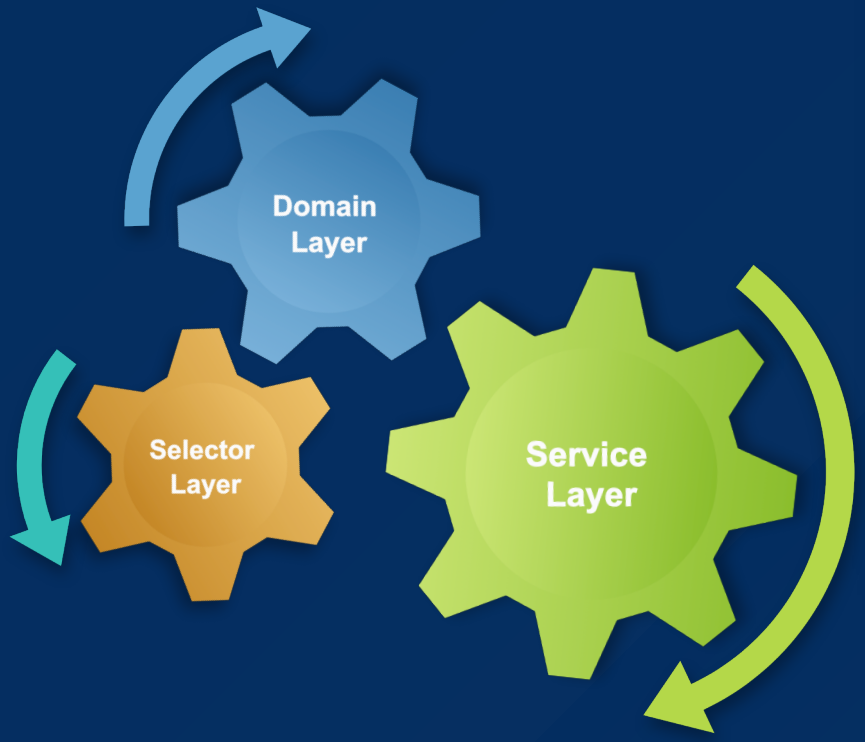
Warehouse Agent



Warehouse Agent deploying Robots!



Service Layers Explained



SERVICE LAYERS EXPLAINED	
Separation of Concerns Recap	✅
The Unit of Work	✅
Service Layer Principles	✅
Warehouse Operations App Overview	✅
Warehouse Operations App Code	✅
Warehouse Operations Agent	✅

What's next in the series

	SESSION
✅	Session #1 - Separation of Concerns in Apex: Why Your Future Self Will Thank You
✅	Session #2 - Service Layers Explained: Coordinating Business Logic in Apex
☐	Session #3 - Domain vs Service: Where Should Your Apex Logic Live?
☐	Session #4 - Query Logic as a First-Class Architecture Concern
☐	Session #5 - Mocking in Apex: Why It Changes Everything
☐	Session #6 - Enterprise-Scale Apex Across Multiple Packages

Thank you!

Andrew Fawcett · Code With Sally

FFLib Series · Session 002